Order Management System

Full Stack Development – TASK 4

Relational Database Operations with JOIN and Subqueries

1 Introduction

The Order Management System (TASK 4) demonstrates how relational databases work in real-world applications such as E-commerce and retail systems.

This system stores customers, products, and orders in separate tables and retrieves related information using SQL JOIN operations. Subqueries are used to identify the highest value order and the most active customer.

This project shows how Node.js and MySQL work together to build a backend system for order processing.

2 Software Requirements

- Node.js
- MySQL Server
- MySQL Workbench
- VS Code
- Web Browser
- Git (Optional)

3 Steps Before Starting the Project

1. Install Node.js and MySQL.
2. Create a database named order_management.
3. Create the required tables: Customers, Products, Orders.
4. Install required packages:

```
npm init -y
npm install express mysql2 dotenv ejs
```

5. Run the server using:

```
node server.js
```

4 Project Structure

```
TASK_4/

 server.js
 package.json
 .env

 config/
    db.js

 public/
    style.css

 views/
     index.ejs
     dashboard.ejs
```

5 Database Design

Database: order_management

5.1 Customers Table

```sql
CREATE TABLE Customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50)
);
```

Listing 1: Customers Table Creation

5.2 Products Table

```sql
CREATE TABLE Products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);
```

Listing 2: Products Table Creation

5.3 Orders Table

```sql
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    quantity INT,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

Listing 3: Orders Table Creation with Foreign Keys

5.4 Sample Data Insertion

```sql
-- Insert sample customers
INSERT INTO Customers (name, email, city) VALUES
('John Doe', 'john@email.com', 'Mumbai'),
('Jane Smith', 'jane@email.com', 'Delhi'),
('Bob Johnson', 'bob@email.com', 'Bangalore');

-- Insert sample products
INSERT INTO Products (product_name, price) VALUES
('Laptop', 55000.00),
('Smartphone', 25000.00),
('Headphones', 2000.00),
('Mouse', 500.00);

-- Insert sample orders
INSERT INTO Orders (customer_id, product_id, quantity, order_date) VALUES
(1, 1, 1, '2024-01-15'),
(2, 2, 2, '2024-01-16'),
(1, 3, 3, '2024-01-17'),
(3, 1, 1, '2024-01-18'),
(2, 4, 5, '2024-01-19');
```

Listing 4: Sample Data for Testing

6 Database Connection Code (config/db.js)

```javascript
const mysql = require("mysql2");
require("dotenv").config();

const db = mysql.createConnection({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME
});

db.connect((err) => {
    if (err) {
        console.log("Database connection failed");
    } else {
        console.log("Connected to MySQL Database");
    }
});

module.exports = db;
```

Listing 5: MySQL Connection Configuration

7 Environment Variables (.env)

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=yourpassword
DB_NAME=order_management
PORT=3000
```

Listing 6: Environment Configuration

8 Main Server Code (server.js)

```
const express = require("express");
const db = require("./config/db");
require("dotenv").config();

const app = express();

app.set("view engine", "ejs");
app.use(express.static("public"));

app.get("/", (req, res) => {
    res.render("index");
});

app.get("/dashboard", (req, res) => {
    const orderHistory = `
        SELECT
            c.name AS customer_name,
            p.product_name,
            o.quantity,
            p.price,
            (o.quantity * p.price) AS total_amount,
            o.order_date
        FROM Orders o
        JOIN Customers c ON o.customer_id = c.customer_id
        JOIN Products p ON o.product_id = p.product_id
        ORDER BY o.order_date DESC
    `;

    const highestOrder = `
        SELECT MAX(o.quantity * p.price) AS highest_value
        FROM Orders o
        JOIN Products p ON o.product_id = p.product_id
    `;

    const activeCustomer = `
        SELECT c.name, COUNT(o.order_id) AS total_orders
        FROM Customers c
        JOIN Orders o ON c.customer_id = o.customer_id
        GROUP BY c.name
        ORDER BY total_orders DESC
        LIMIT 1
    `;

    db.query(orderHistory, (err, orders) => {
        if (err) throw err;

        db.query(highestOrder, (err, highest) => {
            if (err) throw err;

            db.query(activeCustomer, (err, active) => {
                if (err) throw err;

                res.render("dashboard", {
                    orders: orders,
                    highest: highest[0],
                    active: active[0]
                });
            });
        });
    });
});

app.listen(process.env.PORT, () => {
    console.log("Server running on port " + process.env.PORT);
```

```
});
```

Listing 7: Server with JOIN Queries and Subqueries

9 Frontend Code

9.1 views/index.ejs

```
<!DOCTYPE html>
<html>
<head>
    <title>Order Management</title>
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="container">
        <h2>Order Management System</h2>
        <p>Welcome to the Order Management Dashboard</p>
        <a href="/dashboard" class="btn">View Dashboard</a>
    </div>
</body>
</html>
```

Listing 8: Home Page

9.2 views/dashboard.ejs

```
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="/style.css">
    <title>Dashboard</title>
</head>
<body>
    <div class="container">
        <h2>Customer Order History</h2>

        <table>
            <tr>
                <th>Customer</th>
                <th>Product</th>
                <th>Quantity</th>
                <th>Price (  )</th>
                <th>Total (  )</th>
                <th>Date</th>
            </tr>
            <% orders.forEach(order => { %>
            <tr>
                <td><%= order.customer_name %></td>
                <td><%= order.product_name %></td>
                <td><%= order.quantity %></td>
                <td>    <%= order.price %></td>
                <td>    <%= order.total_amount %></td>
                <td><%= order.order_date %></td>
            </tr>
            <% }) %>
        </table>

        <div class="insights">
            <h3>Highest Value Order:     <%= highest.highest_value %></h3>
            <h3>Most Active Customer: <%= active.name %></h3>
        </div>

```

```
        <a href="/" class="btn">Back to Home</a>
    </div>
</body>
</html>
```

Listing 9: Dashboard with Order History and Insights

10 CSS Styling (public/style.css)

```
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    margin: 0;
    padding: 20px;
    min-height: 100vh;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}

h2 {
    color: #333;
    text-align: center;
    margin-bottom: 30px;
}

table {
    width: 100%;
    margin: 20px 0;
    border-collapse: collapse;
    background: white;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

th {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 12px;
    font-weight: bold;
}

td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: center;
}

tr:nth-child(even) {
    background-color: #f8f9fa;
}

tr:hover {
    background-color: #f1f3f5;
}

.insights {
    background: #f8f9fa;
    padding: 20px;
    border-radius: 8px;
```

```
    margin: 20px 0;
    border-left: 4px solid #764ba2;
}

.insights h3 {
    margin: 10px 0;
    color: #333;
}

.btn {
    display: inline-block;
    padding: 10px 20px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    text-decoration: none;
    border-radius: 5px;
    transition: transform 0.3s;
}

.btn:hover {
    transform: translateY(-2px);
}
```

Listing 10: Dashboard Styling

11 SQL Queries Demonstrated

11.1 JOIN Operation

```
SELECT
    c.name AS customer_name,
    p.product_name,
    o.quantity,
    p.price,
    (o.quantity * p.price) AS total_amount,
    o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id
ORDER BY o.order_date DESC;
```

Listing 11: JOIN to Fetch Related Data

11.2 Aggregate Function with JOIN

```
SELECT MAX(o.quantity * p.price) AS highest_value
FROM Orders o
JOIN Products p ON o.product_id = p.product_id;
```

Listing 12: Finding Highest Value Order

11.3 GROUP BY with ORDER BY and LIMIT

```
SELECT c.name, COUNT(o.order_id) AS total_orders
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY total_orders DESC
LIMIT 1;
```

Listing 13: Most Active Customer

12 Features Implemented

- **Relational Database Design**
  - Three related tables with foreign key constraints
  - Proper data normalization
  - Referential integrity
- **JOIN Operations**
  - INNER JOIN to combine customer, product, and order data
  - Calculated fields (quantity × price)
  - Sorted results by date
- **Aggregate Functions**
  - MAX() for highest value order
  - COUNT() with GROUP BY for customer orders
  - Calculated totals for each order
- **Business Insights**
  - Complete order history with customer details
  - Highest value transaction identification
  - Most active customer recognition
- **Responsive Dashboard**
  - Clean table layout
  - Visual insights section
  - Professional gradient styling

13 Testing Scenarios

Scenario 1: View All Orders

- Navigate to dashboard
- Expected: Complete order history with customer and product details
- Verify calculated totals are correct

Scenario 2: Highest Value Order

- Check insights section
- Expected: Maximum order amount displayed
- Verify against manual calculation

Scenario 3: Most Active Customer

- Check insights section
- Expected: Customer with most orders displayed
- Verify by counting orders per customer

14 Expected Output Example

Table 1: Sample Order History Display

Customer	Product	Qty	Price	Total	Date
John Doe	Laptop	1	55000	55000	2024-01-15
Jane Smith	Smartphone	2	25000	50000	2024-01-16
John Doe	Headphones	3	2000	6000	2024-01-17
Bob Johnson	Laptop	1	55000	55000	2024-01-18
Jane Smith	Mouse	5	500	2500	2024-01-19

Insights:

- Highest Value Order: 55000
- Most Active Customer: Jane Smith (2 orders)

15 Steps to Execute

1. **Set up the database:**
   - Open MySQL Workbench
   - Create database: `CREATE DATABASE order_management;`
   - Run the table creation scripts
   - Insert sample data
2. **Configure environment:**
   - Create .env file with database credentials
   - Update password if needed
3. **Install dependencies:**

   ```
   npm install
   ```

4. **Start the server:**

   ```
   node server.js
   ```

5. **Access the application:**

   ```
   http://localhost:3000
   ```

6. **View dashboard:**
   - Click "View Dashboard" link
   - Explore order history and insights

16 Conclusion

TASK 4 demonstrates the use of JOIN operations and subqueries in a full-stack web application. By integrating Node.js with MySQL, the system retrieves relational data efficiently and provides useful insights such as customer order history, highest value order, and most active customer.

Key Learning Outcomes

- Implementing foreign key relationships in MySQL
- Writing complex JOIN queries across multiple tables
- Using aggregate functions for business insights
- Calculating derived fields in SQL queries
- Building a complete full-stack data dashboard

This approach is widely used in E-commerce and retail management systems for generating reports and analyzing customer behavior.